

How My WireGuard + Vultr VPC Lab Actually Worked

Introduction

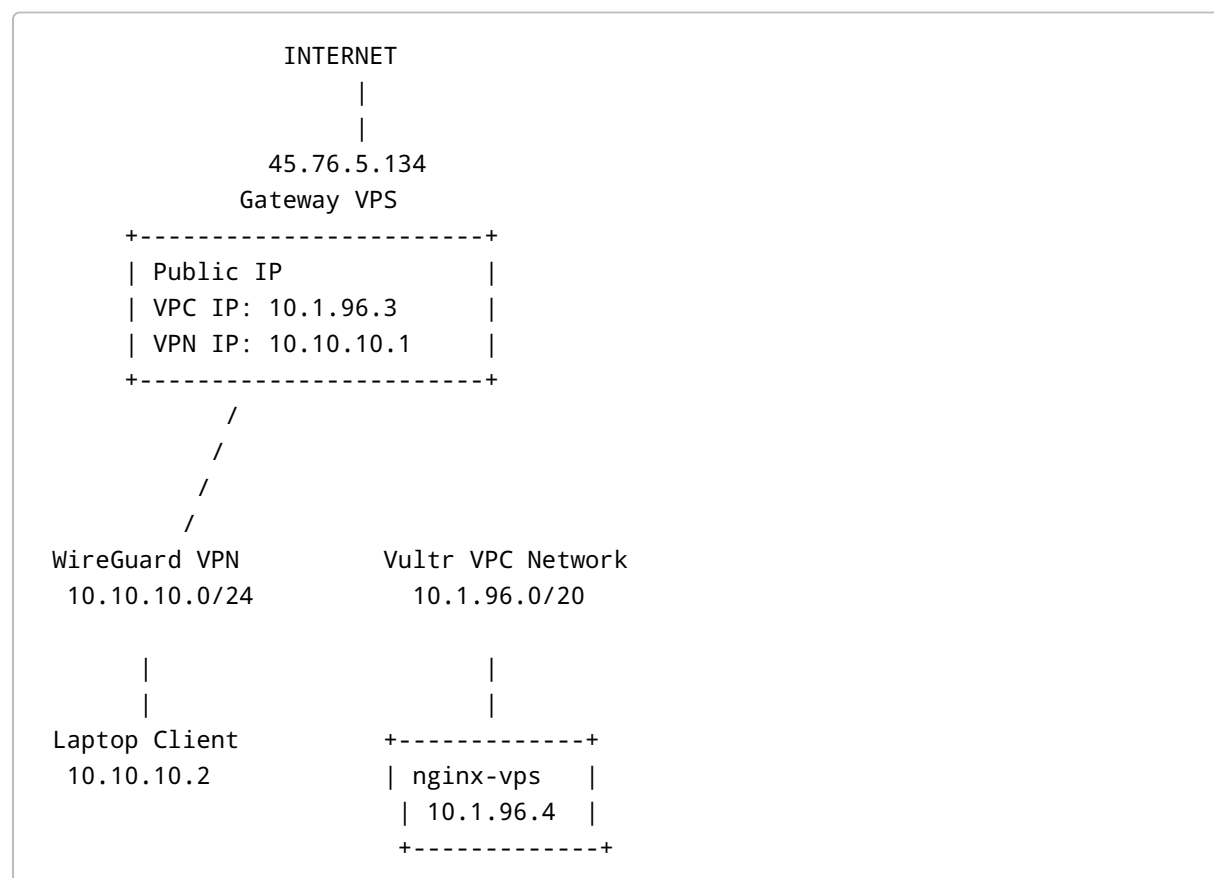
The purpose of this lab was to create a secure private network in the cloud.

Instead of exposing web servers and database servers directly to the Internet, I created a VPN gateway that acted as a secure entry point.

The final architecture contained three different networks:

1. Public Internet
2. WireGuard VPN Network
3. Vultr Private VPC Network

Final Architecture



```
+-----+
| db-vps |
| 10.1.96.5 |
+-----+
```

Understanding Every IP Address

Gateway Public IP

45.76.5.134

Purpose:

This IP exists on the Internet.

My laptop uses this address to establish the VPN tunnel.

Think of this as the front door of the cloud environment.

Gateway VPC IP

10.1.96.3

Purpose:

This IP exists inside the private Vultr network.

Nginx and MariaDB use this IP when communicating with the gateway.

Think of this as the gateway's internal office address.

Gateway VPN IP

10.10.10.1

Purpose:

This IP exists only inside the WireGuard tunnel.

The laptop communicates with this address after the VPN connection is established.

Think of this as the VPN side of the router.

Laptop VPN IP

10.10.10.2

Purpose:

This IP was manually assigned inside WireGuard.

It only exists inside the VPN tunnel.

This is not an Internet address.

Nginx Server

10.1.96.4

Purpose:

Private web server.

Accessible only through VPN.

Runs nginx on TCP port 80.

Database Server

10.1.96.5

Purpose:

Private database server.

Runs MariaDB on TCP port 3306.

Accessible only through VPN.

Where Did 10.10.10.0/24 Come From?

Answer:

I created it.

WireGuard does not automatically assign addresses.

I configured:

Gateway:

Address = 10.10.10.1/24

Laptop:

Address = 10.10.10.2/24

WireGuard then created a virtual network:

10.10.10.0/24

Exactly the same concept as a home router creating:

192.168.1.0/24

Where Did 10.1.96.0/20 Come From?

Answer:

Vultr created it.

When I created the VPC network, Vultr automatically assigned:

10.1.96.0/20

All cloud servers joined this private network.

How Routing Worked

A route means:

"If traffic wants to reach Network X, send it through Device Y."

Example:

Destination: 10.1.96.0/20

Next Hop: WireGuard Tunnel

This route was automatically installed on Windows because of:

AllowedIPs = 10.1.96.0/20

Result:

Whenever I accessed:

10.1.96.4

Windows automatically sent the packet into the VPN tunnel.

Packet Flow Example

I entered:

curl http://10.1.96.4

Step 1

Laptop creates packet:

Source: 10.10.10.2

Destination: 10.1.96.4

Step 2

Windows routing table says:

10.1.96.0/20

Use WireGuard tunnel.

Packet enters VPN.

Step 3

WireGuard encrypts packet.

Packet travels through Internet.

Destination:

45.76.5.134

Step 4

Gateway VPS receives encrypted packet.

WireGuard decrypts packet.

Now Linux sees:

Source: 10.10.10.2

Destination: 10.1.96.4

Step 5

Linux routing table says:

10.1.96.4 is inside VPC.

Send packet through VPC interface.

Step 6

Nginx receives request.

Returns webpage.

Step 7

Response returns to gateway.

Gateway sends response back through VPN.

Step 8

Laptop receives webpage.

Browser displays nginx page.

Success.

Why IP Forwarding Was Required

Default Linux behavior:

Receive traffic only for itself.

Forwarding disabled.

Diagram:

Laptop | Gateway X Nginx

Packet dies.

Command:

```
sysctl -w net.ipv4.ip_forward=1
```

Effect:

Gateway becomes a router.

Diagram:

Laptop | Gateway | Nginx

Traffic flows successfully.

Why NAT Was Required

Without NAT:

Nginx receives:

Source: 10.10.10.2

Destination: 10.1.96.4

Problem:

Nginx does not know where 10.10.10.2 exists.

Response cannot return.

Connection fails.

NAT Command

```
iptables -t nat -A POSTROUTING  
-s 10.10.10.0/24  
-d 10.1.96.0/20  
-j MASQUERADE
```

What MASQUERADE Did

Before NAT

10.10.10.2 --> 10.1.96.4

After NAT

10.1.96.3 --> 10.1.96.4

The gateway rewrites the source address.

Now nginx thinks traffic came from:

10.1.96.3

which belongs to the VPC.

Responses return successfully.

Why UFW Blocked Traffic

UFW had:

deny (routed)

Meaning:

Gateway can receive traffic.

Gateway can send traffic.

Gateway cannot forward traffic.

Fix:

ufw route allow in on wg0 out on enp8s0

Meaning:

Allow VPN traffic to leave through the VPC interface.

Why Two Firewalls Existed

Firewall Layer 1

Vultr Firewall Group

Cloud-level firewall.

Traffic blocked before reaching the VPS.

Internet X Server

Firewall Layer 2

UFW

Host-level firewall.

Traffic already reached Linux.

Internet | Server | UFW

Think:

Building Gate

+ Apartment Door

Two independent security layers.

Validation Performed

VPN Tunnel

ping 10.10.10.1

Success

Nginx Reachability

ping 10.1.96.4

Success

HTTP Connectivity

curl http://10.1.96.4

Success

Database Reachability

Test-NetConnection 10.1.96.5 -Port 3306

TcpTestSucceeded : True

Success

Final Summary

The laptop established a WireGuard VPN connection to the gateway's public IP.

The gateway connected both the VPN network and the private Vultr VPC.

IP forwarding transformed Linux into a router.

NAT translated VPN addresses into VPC addresses.

Firewall rules allowed secure communication.

The result was a private cloud environment where nginx and MariaDB remained inaccessible from the public Internet while still being reachable through a secure VPN connection.